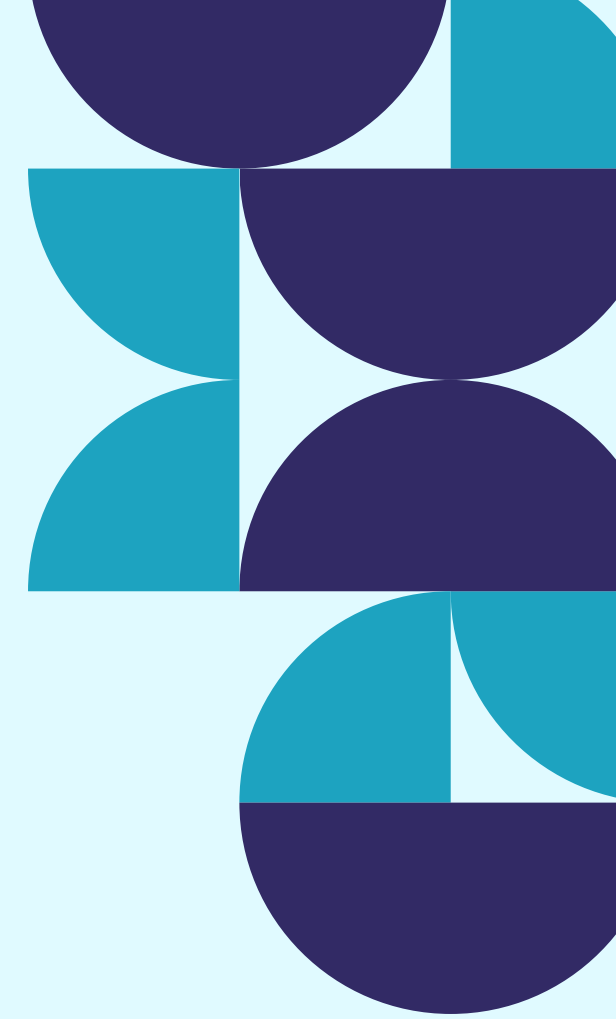


PREDICTING MAXIMUM VON MISES STRESS

The goal is to predict the maximum Von Mises stress in a cantilever beam from its thickness distribution, and compare two approaches one using raw data, one using dimensionality reduction.

Presented By Shivam Badyal, Srajan Sijariya

TWO STRATEGIES FOR PREDICTION



NEURAL NETWORK

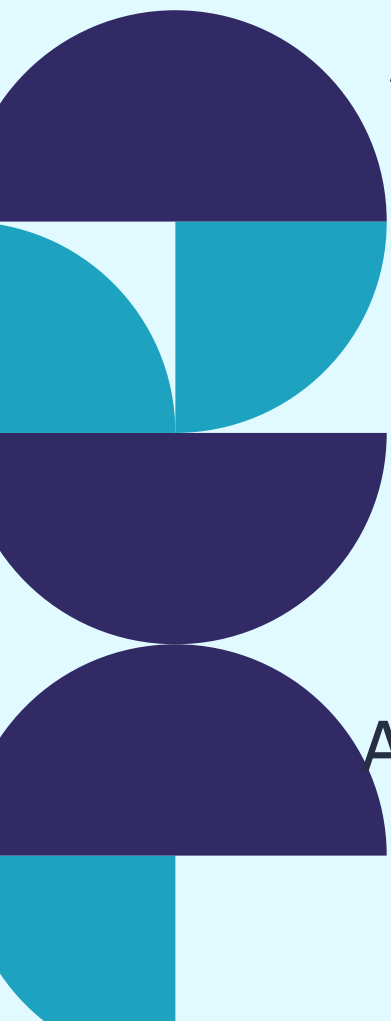
APPROACH 1: [226 thickness values] —► [MLP: 512→256→128→64→1] —► [Max Stress]

- All 226 features fed directly
 - Larger network needed
 - More parameters to train

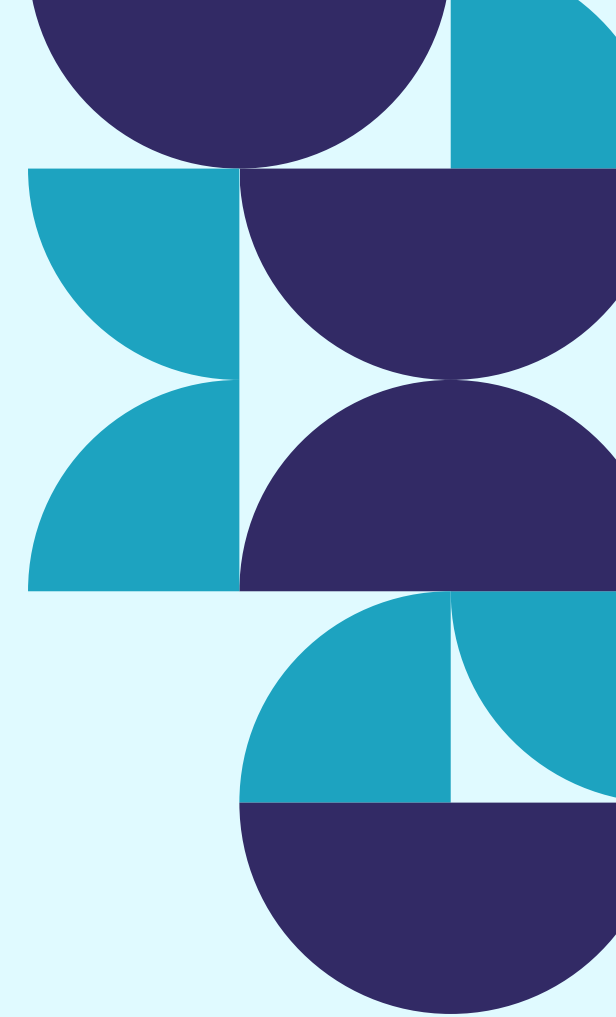
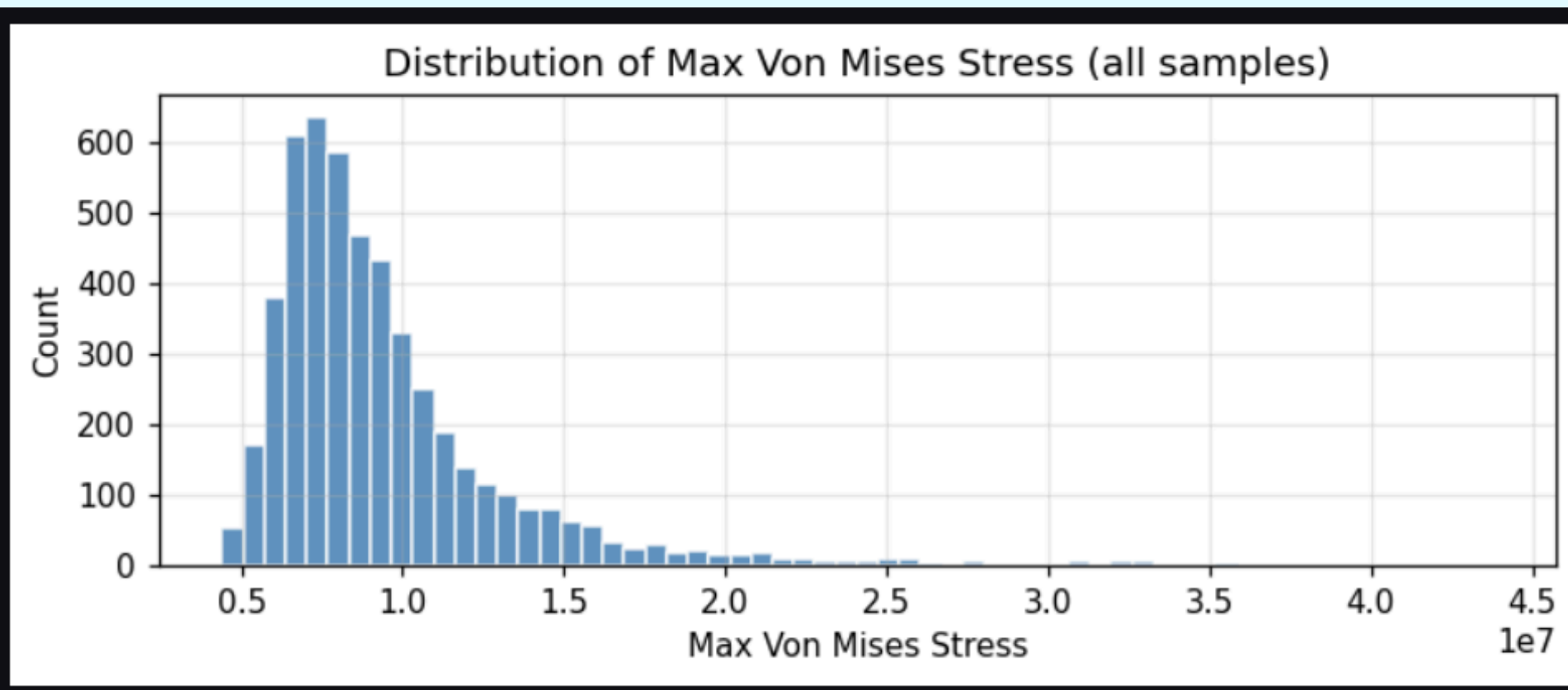
NEURAL NETWORK + PCA

APPROACH 2: [226 thickness values] —► [PCA: 226→208 dims] —► [MLP: 256→128→64→1] —► [Max Stress]

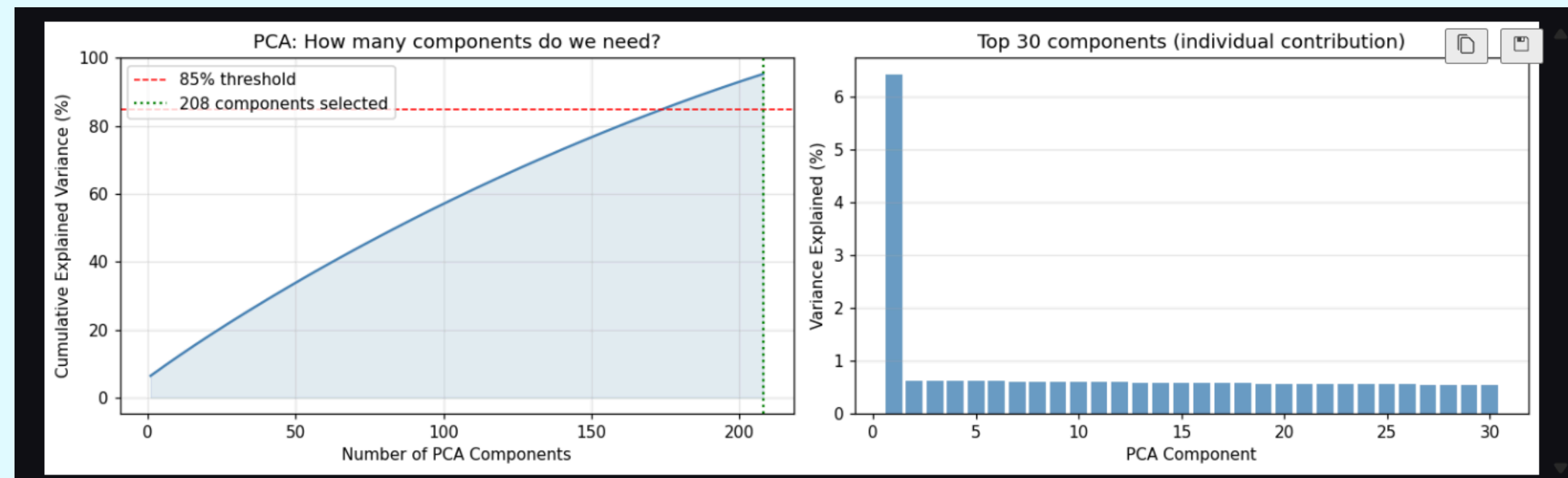
- First compress features using PCA
 - Smaller, simpler network
 - Hypothesis: needs LESS DATA



GIVEN DATA VISULIZATION



Given Data is very skewed



APPROACH 1 TRAINING

```
Epoch 1/1000
107/107 3s 6ms/step - loss: 5.6633 - mae: 1.8468 - val_loss: 0.9957 - val_mae: 0.6717 - learning_rate: 5.0000e-04
Epoch 2/1000
107/107 1s 5ms/step - loss: 2.6236 - mae: 1.2490 - val_loss: 0.9017 - val_mae: 0.6118 - learning_rate: 5.0000e-04
Epoch 3/1000
107/107 1s 5ms/step - loss: 1.9396 - mae: 1.0550 - val_loss: 0.8398 - val_mae: 0.5816 - learning_rate: 5.0000e-04
Epoch 4/1000
107/107 0s 4ms/step - loss: 1.6470 - mae: 0.9607 - val_loss: 0.7895 - val_mae: 0.5260 - learning_rate: 5.0000e-04
Epoch 5/1000
107/107 1s 5ms/step - loss: 1.4086 - mae: 0.8824 - val_loss: 0.7312 - val_mae: 0.5006 - learning_rate: 5.0000e-04
Epoch 6/1000
107/107 1s 5ms/step - loss: 1.2606 - mae: 0.8319 - val_loss: 0.6878 - val_mae: 0.4858 - learning_rate: 5.0000e-04
Epoch 7/1000
107/107 1s 5ms/step - loss: 1.1643 - mae: 0.7943 - val_loss: 0.6419 - val_mae: 0.4544 - learning_rate: 5.0000e-04
Epoch 8/1000
107/107 1s 5ms/step - loss: 1.0210 - mae: 0.7376 - val_loss: 0.5981 - val_mae: 0.4476 - learning_rate: 5.0000e-04
Epoch 9/1000
107/107 1s 5ms/step - loss: 0.8977 - mae: 0.6883 - val_loss: 0.5601 - val_mae: 0.4251 - learning_rate: 5.0000e-04
Epoch 10/1000
107/107 1s 5ms/step - loss: 0.8348 - mae: 0.6622 - val_loss: 0.5324 - val_mae: 0.4234 - learning_rate: 5.0000e-04
Epoch 11/1000
107/107 1s 5ms/step - loss: 0.7732 - mae: 0.6330 - val_loss: 0.4963 - val_mae: 0.4104 - learning_rate: 5.0000e-04
Epoch 12/1000
107/107 1s 5ms/step - loss: 0.6737 - mae: 0.5878 - val_loss: 0.4866 - val_mae: 0.4138 - learning_rate: 5.0000e-04
Epoch 13/1000
...
Epoch 154: early stopping
Restoring model weights from the end of the best epoch: 114.

🔌 Finished after 154 epochs in 62.7s
```

because of validation data early epoch stop

If val_loss does not improve for 40 consecutive epochs

STOP Approach 1: Best val_loss reached at epoch ~134 val_loss increased for 20 epochs after that →

Stopped at epoch 154 Approach

2: Best val_loss reached at epoch ~108 val_loss increased for 20 epochs after that → Stopped at epoch 128

APPROACH 2 TRAINING

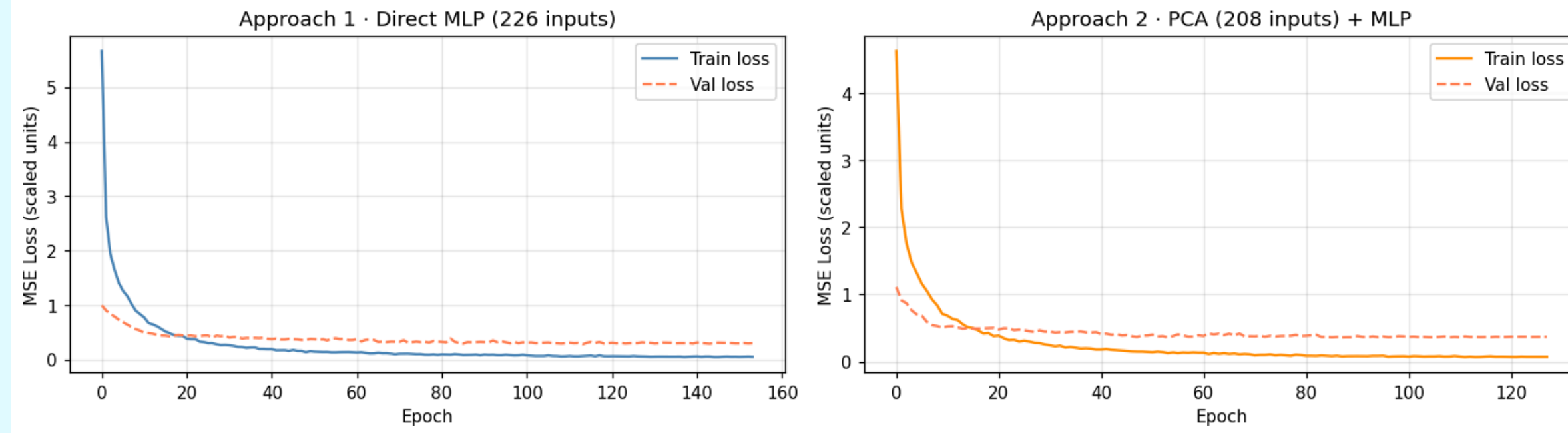
```
Epoch 1/1000
107/107 2s 4ms/step - loss: 4.6277 - mae: 1.6333 - val_loss: 1.1109 - val_mae: 0.6634 - learning_rate: 5.0000e-04
Epoch 2/1000
107/107 0s 3ms/step - loss: 2.2808 - mae: 1.1515 - val_loss: 0.9115 - val_mae: 0.5669 - learning_rate: 5.0000e-04
Epoch 3/1000
107/107 0s 3ms/step - loss: 1.7580 - mae: 1.0026 - val_loss: 0.8680 - val_mae: 0.5334 - learning_rate: 5.0000e-04
Epoch 4/1000
107/107 0s 3ms/step - loss: 1.4749 - mae: 0.9007 - val_loss: 0.7619 - val_mae: 0.4943 - learning_rate: 5.0000e-04
Epoch 5/1000
107/107 0s 3ms/step - loss: 1.3204 - mae: 0.8458 - val_loss: 0.7030 - val_mae: 0.4709 - learning_rate: 5.0000e-04
Epoch 6/1000
107/107 0s 3ms/step - loss: 1.1599 - mae: 0.7872 - val_loss: 0.6836 - val_mae: 0.4530 - learning_rate: 5.0000e-04
Epoch 7/1000
107/107 0s 3ms/step - loss: 1.0506 - mae: 0.7427 - val_loss: 0.5904 - val_mae: 0.4336 - learning_rate: 5.0000e-04
Epoch 8/1000
107/107 0s 3ms/step - loss: 0.9248 - mae: 0.6977 - val_loss: 0.5418 - val_mae: 0.4133 - learning_rate: 5.0000e-04
Epoch 9/1000
107/107 0s 3ms/step - loss: 0.8373 - mae: 0.6564 - val_loss: 0.5293 - val_mae: 0.4101 - learning_rate: 5.0000e-04
Epoch 10/1000
107/107 0s 3ms/step - loss: 0.7103 - mae: 0.6115 - val_loss: 0.5057 - val_mae: 0.4076 - learning_rate: 5.0000e-04
Epoch 11/1000
107/107 0s 3ms/step - loss: 0.6836 - mae: 0.5915 - val_loss: 0.5238 - val_mae: 0.4003 - learning_rate: 5.0000e-04
Epoch 12/1000
107/107 0s 3ms/step - loss: 0.6349 - mae: 0.5628 - val_loss: 0.5276 - val_mae: 0.3953 - learning_rate: 5.0000e-04
Epoch 13/1000
...
Epoch 128: early stopping
Restoring model weights from the end of the best epoch: 88.

🔌 Finished after 128 epochs in 47.3s
```

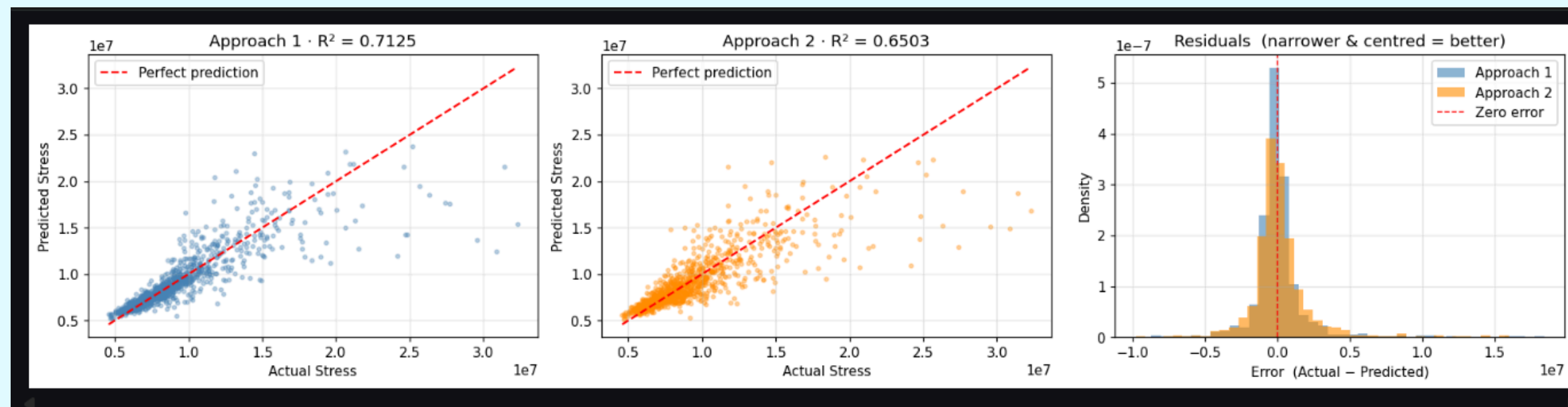
because of validation data early epoch stop

LEARNING CURVE

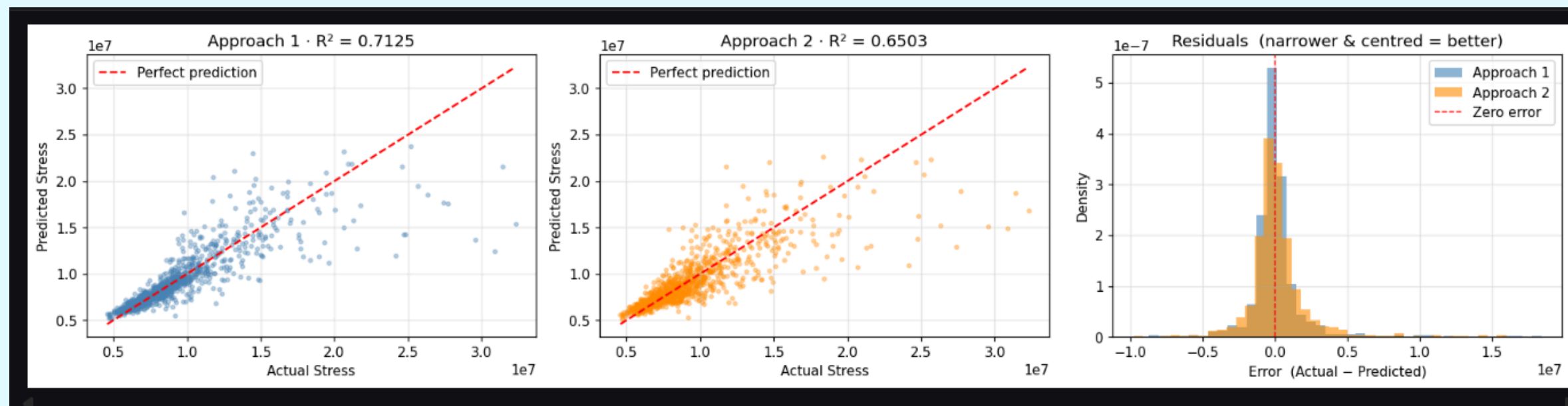
Learning Curves — did the model learn steadily?



PREDICTED VS ACTUAL & RESIDUALS

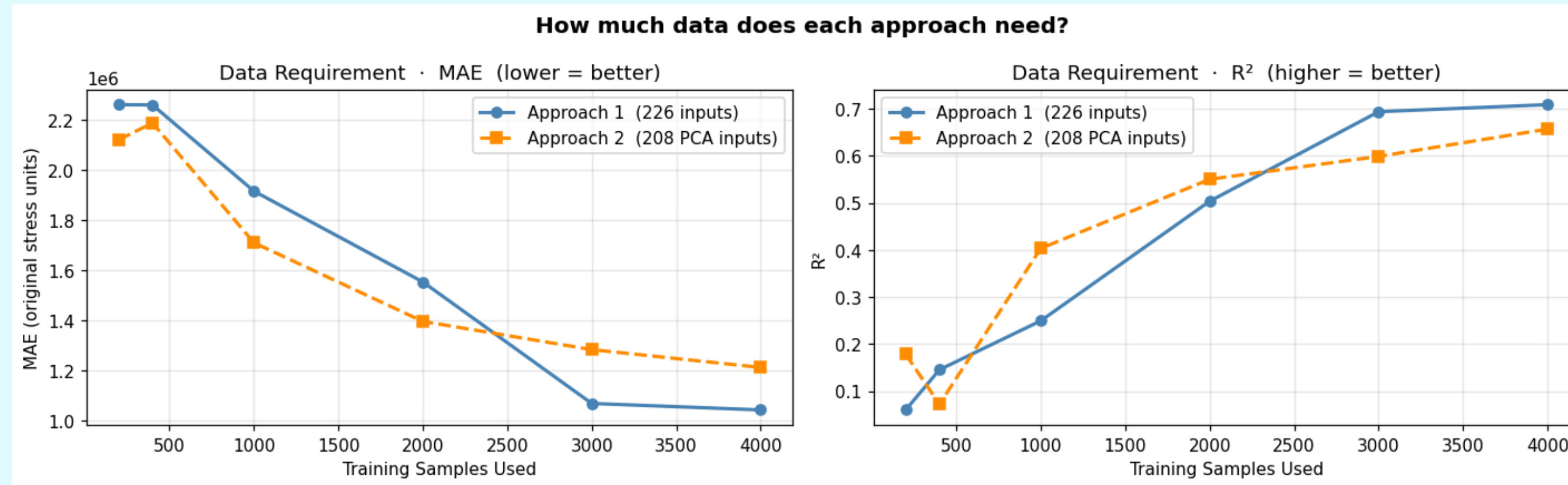


WHY RESULTS ARE SCATTERED?



Dataset distribution: Most samples → low-medium stress (model sees many, learns well) Few samples → very high stress (model sees few, learns poorly) High stress = rare extreme cases → Network hasn't seen enough of them to learn the pattern

RESULT



App1 (Direct MLP) wins because:

- With enough data, the larger network learns better
- The 18 features PCA discarded actually contained useful info
- Approach 1 recovers that lost information
- More data = more benefit from more capacity

App2 (PCA + smaller network) wins because:

- Fewer parameters to train → Less data needed to fit the model → PCA acts as a regularizer even with minimal compression
- 208 dims still slightly simpler than 226 dims

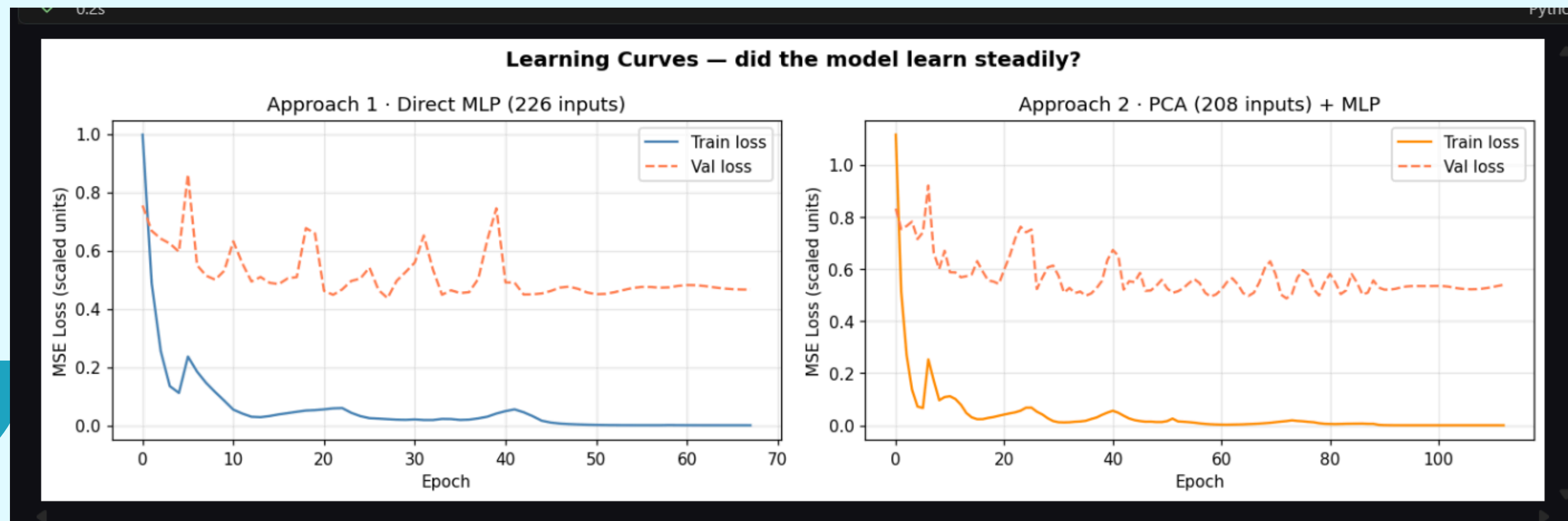
SUMMARY

Final Comparison: Approach 1 vs Approach 2		
	Approach 1 (Direct MLP)	Approach 2 (PCA + MLP)
Property		
Input dimensions	226	208
Network architecture	512 → 256 → 128 → 64 → 1	256 → 128 → 64 → 1
MAE (original units)	1015443.8125	1247554.8750
RMSE (original units)	1955446.0000	2156423.5000
R^2	0.7125	0.6503
Training epochs	154	128

INITIAL ERRORS

OverFitting

Train loss → goes to nearly 0.0 ✓
Val loss → stays stuck at 0.5 ✗



OPTIMISATIONS WE DID TO REDUCE OUR INITIAL ERRORS:-

1) We standardised the given thickness inputs

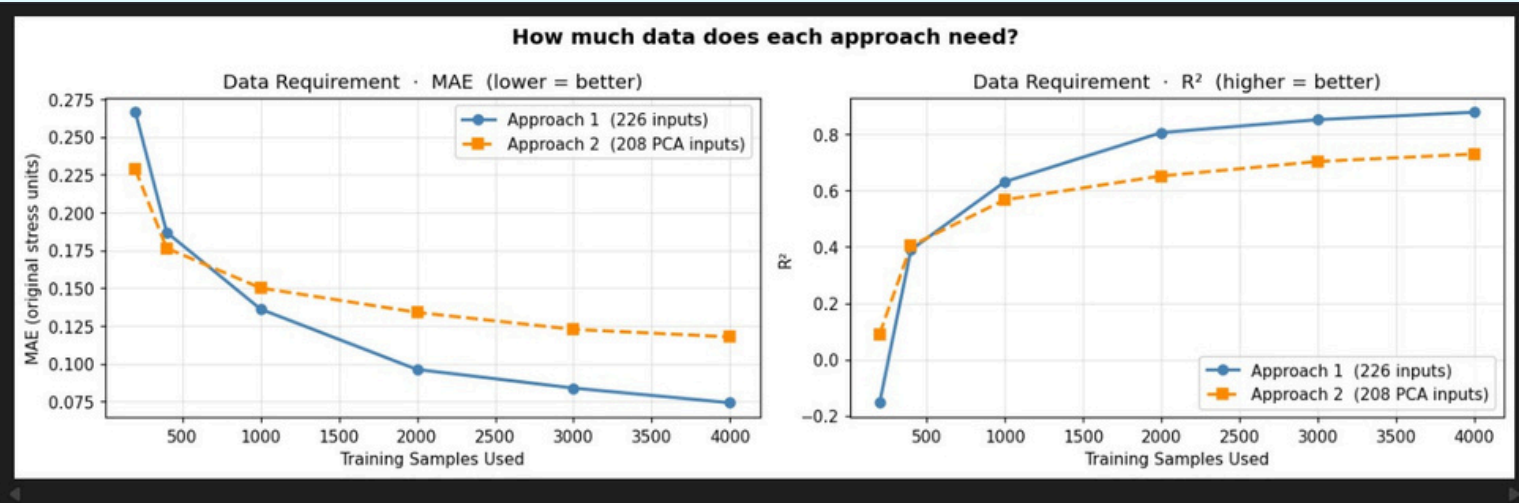
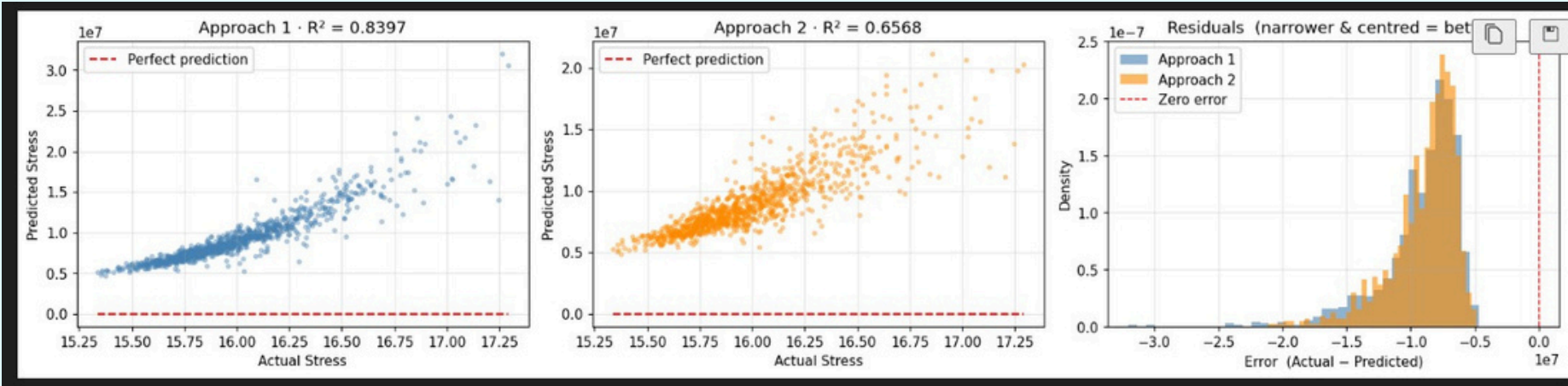
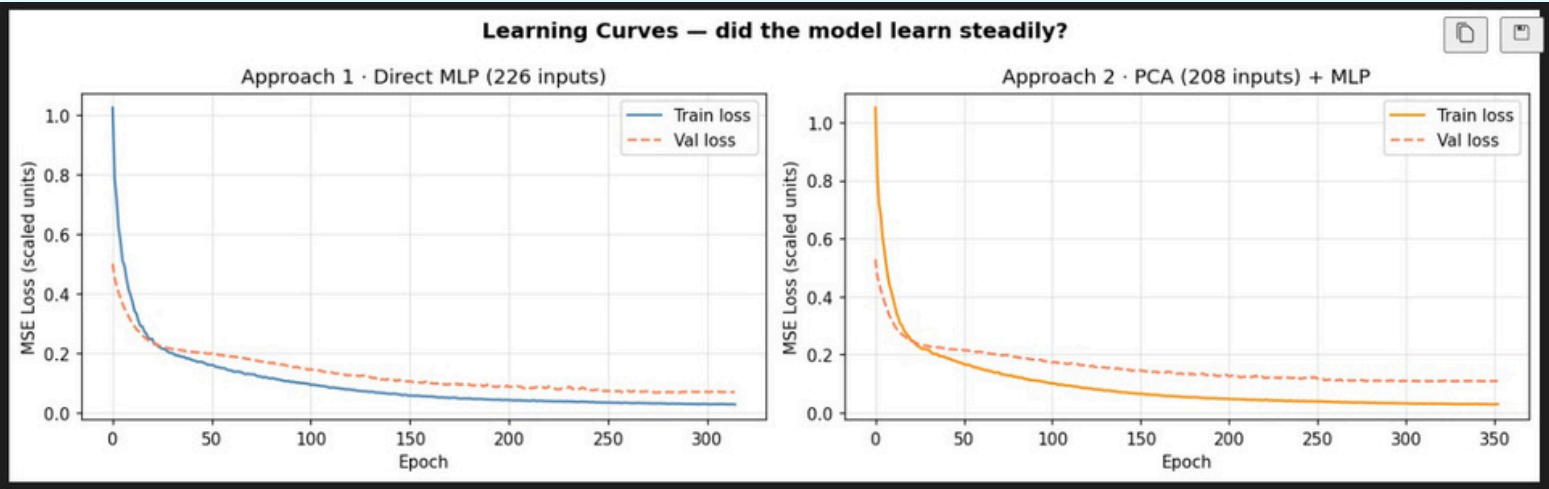
2) We took y as $\log(1+y)$ instead of standard y to reduce skewness for better training

3) We also implemented callbacks to reduce overfitting and we also reduced learning rate if val loss plateau

OPTIMISATIONS WE DID USING GEN AI:-

- 1) We changed the activation function to Swish activation function and Huber loss function
- 2) Also used Dropout method to reduce dead neurons in the model
- 3) implemented K cross fold validation to check the data on unique combination of data sets instead of a single data set
- 4) after taking $\log(1+y)$ we also did quantile transform to further get the bell shaped curve for even better training and optimisation

RESULTS WE GOT AFTER DOING ALL OPTIMISATIONS(INCLUDING GEN AI ONES):-



Final Comparison: Approach 1 vs Approach 2		
	Approach 1 (Direct MLP)	Approach 2 (PCA + MLP)
Property		
Input dimensions	226	208
Network architecture	512 → 256 → 128 → 64 → 1	256 → 128 → 64 → 1
MAE (original units)	714064.5000	1177882.3750
RMSE (original units)	1372231.5000	2080982.6250
R^2	0.8397	0.6568
Training epochs	315	353

